Sketch First, Build Second

5 ASCII Planning Prompts for Claude Code
Copy-paste ready. Works for dashboards, slides, workflows, schemas, and landing pages.

01 SaaS Analytics Dashboard

02 PowerPoint Deck (10 slides)

03 n8n Workflow Automation

04 Database Schema (Supabase)

05 SaaS Landing Page

++ Master Template (use for anything)

By Mark Kashef • promptadvisers.com • 2026

How to Use This File

Each example has **two versions** you can run side by side:

① **The 3-Turn Sketch Sequence** — Generate → Iterate → Build. This is the technique. Run all 3 turns in order. You'll see the wireframe appear, tweak it for free, then hand it off as a spec.

② **Without vs With comparison** — Run the 'Without Sketch' prompt first to see what Claude decides for you. Then run the 'With Sketch' prompt to see first-try accuracy. The gap is the whole point.

The 3-Step Technique

Step 1: Sketch, don't describe — ask for an ASCII wireframe before any code
Step 2: Iterate on the sketch, not the build — changes here are free
Step 3: Annotate and hand off — "Build this exactly. The wireframe is the specification."

#01

SaaS Analytics Dashboard

Your primary demo. Web UI with sidebar, stat cards, charts, and data table.

SKETCH SEQUENCE

TURN 1 — Generate the wireframe

```
Before writing any code, create an ASCII wireframe of a SaaS analytics dashboard — sidebar,
stat cards, two charts side by side, and a data table below. No code yet.
```

TURN 2 — Iterate (free changes)

```
Two changes only:
1. Make the line chart feel noticeably wider than the pie chart
2. Status column: ● Active (green implied), ■ Inactive (grey implied)
Redraw the full wireframe. Nothing else changes.
```

TURN 3 — Annotate + Build

```
My annotations:
① Sidebar collapsible on mobile — add hamburger toggle at top
② Line chart uses brand blue (#2563EB)
③ [+ New] opens a slide-in drawer from the right

Build this as a React app with Tailwind CSS using the wireframe as the exact specification.
Mock data. Spin it up on localhost:3001.
```

#01 SaaS Analytics Dashboard — Build Comparison

WITHOUT SKETCH — what most people type

```
Build me a SaaS dashboard. I want a sidebar, some stat cards, a couple of charts, and a data
table. React and Tailwind.
```

Watch for: *Charts will probably be stacked or equal width. Sidebar might be 25-30% of the screen. Table columns will be whatever Claude picks. The [+ New] button probably won't exist. All fine — just not what you pictured.*

WITH SKETCH — exact specification

```
Build this dashboard exactly. The ASCII wireframe is the complete specification — every
layout decision is already made. Do not invent anything.

■■■
```

#02

PowerPoint Deck (10 slides)

The non-developer unlock. ASCII planning a full deck shows narrative structure, slide variety, and flow.

SKETCH SEQUENCE

TURN 1 — Wireframe the full deck structure

```
Before making any slides, create an ASCII wireframe for a 10-slide deck about replacing
manual reporting with automated dashboards. Show each slide as a labelled box with a rough
layout. No PowerPoint yet.
```

TURN 2 — Adjust the narrative

```
Two changes:
1. Slide 6 (transition) feels weak — make it a dramatic quote slide instead. Big pull quote
centred, dark background implied.
2. Slide 9 (ROI) — add a fourth row: "Errors: frequent → zero"

Redraw just those two slides.
```

TURN 3 — Build

```
Build this 10-slide PowerPoint using the pptx skill. Wireframe is the exact specification —
vary each slide layout, don't use one template for everything. Topic: manual reporting →
automated dashboards. Stat: 3.2 hrs/week wasted. Quote: "Your team deserves better than a
Friday afternoon spreadsheet." ROI: 3hrs → 0, frequent errors → zero. CTA: schedule a demo.
```

#02 PowerPoint Deck (10 slides) — Build Comparison

WITHOUT SKETCH — what most people type

```
Make me a 10-slide presentation about replacing manual reporting with automated dashboards.
```

Watch for: *Every slide has the same two-column layout or title + bullets. No transition slide, no dramatic stat slide, no split visual slides. The deck looks uniform and generic.*

WITH SKETCH — exact specification

```
Build a 10-slide PowerPoint using the pptx skill. The wireframe below defines the exact
layout of every slide — vary each one:

SLIDE 1 (Title): Logo top-left, big centred title, subtitle, presenter + date bottom
SLIDE 2 (Stat): Headline + giant centred callout box with "3.2 hrs/week per person"
SLIDE 3 (Old Way): Title + 3 equal icon columns: ■ Export / ■ Email / ■ Miss it
SLIDE 4 (Pain 1): 50/50 split — screenshot left, 3 bullet callouts right
SLIDE 5 (Pain 2): Same 50/50 split, different visual + callouts
SLIDE 6 (Transition): Dark background, single centred pull quote, nothing else
SLIDE 7 (New Way): Title + 3 equal icon columns: ■ Auto-refresh / ■ Slack ping / ■ In sync
SLIDE 8 (Dashboard): Large screenshot top 70%, 3 numbered callouts below
SLIDE 9 (ROI): Clean 2-column table: BEFORE vs AFTER, 4 rows
SLIDE 10 (CTA): 3 numbered next steps, CTA button, contact info

Every slide layout must be different. Match the structure above exactly.
Content: manual reporting → automated dashboards. Quote slide: "Your team deserves better
than a Friday afternoon spreadsheet."
```

#03

n8n Workflow Automation

Claude won't just get the layout wrong — it'll skip entire branches. The ASCII makes every branch visible.

SKETCH SEQUENCE

TURN 1 — Generate

```
Before building anything in n8n, create an ASCII flowchart of a lead qualification workflow —
webhook in, validate, CRM lookup, score check, route to sales or nurture. No code yet.
```

TURN 2 — Fix a gap

```
Both branches should end symmetrically with Log to DB → END. Confirm the nurture branch has
this. Also add "(HubSpot POST)" beside Create New Contact. Redraw.
```

TURN 3 — Build

```
Build this as an n8n workflow JSON I can import directly. Every branch exactly as shown.
Placeholder credentials.
```

#03 n8n Workflow Automation — Build Comparison

WITHOUT SKETCH — what most people type

```
Build an n8n workflow. Webhook comes in, validate it, look up the contact in HubSpot, check
their score, and route them to sales or nurture.
```

Watch for: *No error handling on invalid input. Probably no 'contact not found' branch. Log to DB missing entirely. Claude builds the happy path.*

WITH SKETCH — exact specification

```
Build this as n8n workflow JSON. Complete specification:

[WEBHOOK] → [Parse & Validate]
INVALID → [Send Error Email] → END
VALID → [Lookup Contact in CRM]
NOT FOUND → [Create New Contact (HubSpot POST)] → merge
FOUND → merge
→ [Check Lead Score]
< 50 → [Add to Nurture Sequence] → [Log to DB] → END
≥ 50 → [Notify Sales Slack] → [Create CRM Task] → [Log to DB] → END

Every branch present. No missing paths.
Nodes: Webhook, Code JS, HTTP Request, IF, Slack, Supabase. Placeholder credentials.
```

#04

Database Schema (Supabase)

Prevents the worst thing: discovering your schema is wrong after you've built your API against it.

SKETCH SEQUENCE

TURN 1 — Generate

```
Before writing any SQL, create an ASCII ER diagram for a SaaS app — users, products,
purchases, and audit logs. Show the relationships. No SQL yet.
```

TURN 2 — Add a table mid-sketch

```
I need a subscriptions table too: id PK, user_id FK→users, product_id FK→products, status,
started_at, ends_at.
Add it with proper arrows. Redraw full diagram.
```

TURN 3 — Build

```
Write the Supabase SQL migration using this schema as the exact specification. UUID PKs, RLS
on user-owned tables, indexes on FK + status columns. Single migration file.
```

#04 Database Schema (Supabase) — Build Comparison

WITHOUT SKETCH — what most people type

```
Write a Supabase SQL migration for a SaaS app with users, products, and purchases. Add RLS.
```

Watch for: *Subscriptions table missing entirely. Audit logs probably generic or absent. RLS policies too broad or wrong tables.*

WITH SKETCH — exact specification

```
Write a Supabase SQL migration using this schema as the exact specification:

■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■
■ users ■ ■ products ■
■ id (PK) ■ ■ id (PK) ■
■ email, name ■ ■ name, price ■
■ plan ■ ■ stripe_price_id ■
■ stripe_id ■ ■ active ■
■ created_at ■ ■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■ purchases ■■■■■■■■■ ■ subscriptions ■
■ id PK ■■■■■■■■■■■■■■■■■ id PK ■
■ user_id FK ■ ■ user_id FK ■
■ product_id FK ■ ■ product_id FK ■
■ amount, currency ■ ■ status ■
■ status ■ ■ started_at ■
■■■■■■■■■■■■■■■■■■■■■■ ■ ends_at ■
▼ ■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■
■ audit_logs ■
■ id PK ■
■ purchase_id FK ■
■ event_type ■
■ payload (jsonb) ■
■■■■■■■■■■■■■■■■■■■■■■

UUID PKs, RLS users see only their rows, indexes on FK+status, CASCADE delete audit_logs,
single migration file.
```

#05

SaaS Landing Page

The most visual example. Claude will build something that looks good — but it won't match your vision.

SKETCH SEQUENCE

TURN 1 — Generate

```
Before writing any code, create an ASCII wireframe for a SaaS landing page — navbar, hero
with a product screenshot, social proof bar, features, and footer. No code yet.
```

TURN 2 — Iterate

```
Add a pricing section between features and footer: three pricing tier cards side by side
(Free / Pro / Enterprise). Each card has: tier name, price, 3 bullet features, CTA button.
Pro tier is highlighted. Redraw.
```

TURN 3 — Build + run

```
Build this as a landing page for an AI writing tool called Drafter. Use the wireframe as the
exact specification. Tailwind CSS CDN. Spin it up on localhost:3004.
```

#05 SaaS Landing Page — Build Comparison

WITHOUT SKETCH — what most people type

```
Build me a landing page for an AI writing tool called Drafter. Make it modern and clean with a
hero, features, pricing, and footer.
```

Watch for: *Hero section is probably full-width, not a 50/50 split. Features might be 3 columns not 4. Pricing tiers look identical — no visual emphasis on Pro.*

WITH SKETCH — exact specification

```
Build a single-file HTML landing page for "Drafter" using Tailwind CSS CDN. The wireframe
below is the exact specification — match every section:

■■■
```

++

Master Template

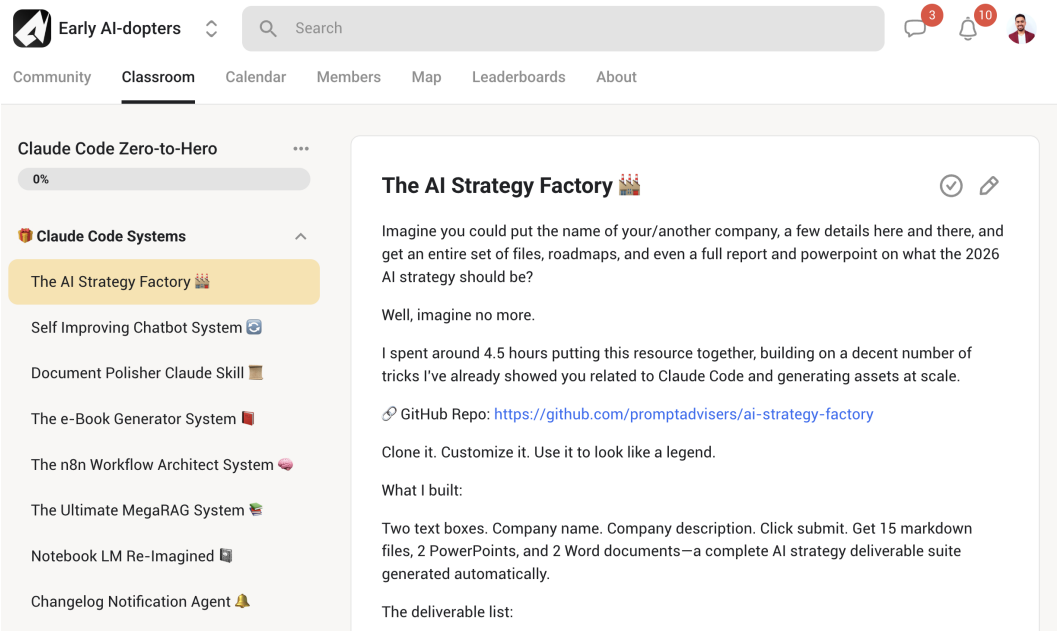
Use this for anything. Swap [thing] with whatever you're building.

STEP 1 — Generate

```
Before writing any code, generate a detailed ASCII wireframe of [thing].
Use box-drawing characters (■ ■ ■ ■ ■ ■ ■ ■) and arrows for flow.
Do not write any code. Output only the ASCII.
```

STEP 2 — Iterate

```
[1-2 specific changes]. Redraw. Nothing else changes.
```

STEP 3 — Build

```
Build [the thing] using this wireframe as the exact specification:

[paste wireframe]

[stack + requirements]
Match the wireframe exactly. Every layout decision is already made.
```